

TP1 - Factory Method (Patrones Creacionales)

Materia: Metodología de Sistemas II

Unidad 2: Patrones de Diseño

Alumno: Nicolás Viruel

Lenguaje: Python (sin librerías externas)

1. Problema identificado

El ReportService mezclaba dos responsabilidades: decidir qué clase instanciar y usar el reporte generado. Cada formato nuevo (PDF, Excel, CSV, HTML) obligaba a modificar el servicio, violando el principio de responsabilidad única.

2. Diagrama UML - ANTES

ReportService

- Método: generate(data, format_type)
- Instancia directamente: PDFReport, ExcelReport o CSVReport según el parámetro

Problema: ReportService conoce e instancia clases concretas. Agregar HTMLReport obliga a editar ReportService.

3. Diagrama UML - DESPUÉS

Interfaz Report (producto)

- Métodos: set_data, add_header, add_footer, render, get_output

Productos concretos (implementan Report)

- PDFReport, ExcelReport, CSVReport, HTMLReport (nuevo - sin tocar ReportService)

ReportFactory (decisión de construcción)

- Método estático: create(format_type) → Report

ReportService (agnóstico al formato)

- Usa ReportFactory.create() y opera solo sobre la interfaz Report
- No instancia ninguna clase concreta

Extensión: agregar HTMLReport solo modifica ReportFactory. ReportService no cambia.

4. Código ANTES (problemático)

```
class ReportService:
    def generate(self, data, format_type):
        # Decision de construccion mezclada con logica de uso
        if format_type == "pdf":
            report = PDFReport()
        elif format_type == "excel":
            report = ExcelReport()
        elif format_type == "csv":
            report = CSVReport()
        else:
            raise ValueError("Formato no soportado")
```

```

report.set_data(data)
report.add_header("Reporte Mensual")
report.add_footer("Generado el " + str(date.today()))
report.render()
return report.get_output()

```

Problema: el servicio importa e instancia PDFReport, ExcelReport, CSVReport directamente.

5. Código DESPUÉS (refactorizado completo)

"""

Variante elegida: Static Factory (Simple Factory).

Factory Method vs Abstract Factory:

Solo hay un producto (Report) con variantes por formato.

Abstract Factory aplicaria si debieramos crear familias coordinadas (ej. Report + Template + Exporter del mismo tema visual).

Aca no hay familias relacionadas; Static Factory es suficiente.

"""

```

from abc import ABC, abstractmethod
from datetime import date

```

```

class Report(ABC):
    @abstractmethod
    def set_data(self, data): ...
    @abstractmethod
    def add_header(self, text): ...
    @abstractmethod
    def add_footer(self, text): ...
    @abstractmethod
    def render(self): ...
    @abstractmethod
    def get_output(self): ...

```

```

class _BaseReport(Report):
    def __init__(self, tag):
        self._tag = tag
        self._data = []
        self._header = ""
        self._footer = ""
        self._rendered = False

    def set_data(self, data):
        self._data = data

    def add_header(self, text):
        self._header = text

    def add_footer(self, text):
        self._footer = text

    def render(self):
        self._rendered = True

```

```

def get_output(self):
    body = ", ".join(str(row) for row in self._data)
    return f"[{self._tag}] {self._header} | {body} | {self._footer}"

class PDFReport(_BaseReport):
    def __init__(self):
        super().__init__("PDF")

class ExcelReport(_BaseReport):
    def __init__(self):
        super().__init__("EXCEL")

class CSVReport(_BaseReport):
    def __init__(self):
        super().__init__("CSV")

class HTMLReport(_BaseReport):
    """Cuarto formato agregado sin modificar ReportService."""

    def __init__(self):
        super().__init__("HTML")

class ReportFactory:
    @staticmethod
    def create(format_type):
        match format_type.lower():
            case "pdf":
                return PDFReport()
            case "excel":
                return ExcelReport()
            case "csv":
                return CSVReport()
            case "html":
                return HTMLReport() # <-- unico cambio para soportar HTML
            case _:
                raise ValueError(f"Formato no soportado: {format_type}")

class ReportService:
    def generate(self, data, format_type):
        report = ReportFactory.create(format_type) # agnostico al formato
        report.set_data(data)
        report.add_header("Reporte Mensual")
        report.add_footer(f"Generado el {date.today().isoformat()}")
        report.render()
        return report.get_output()

```

ReportService agnóstico: no importa ni instancia ninguna clase concreta. Solo conoce ReportFactory y la interfaz Report.

HTMLReport sin tocar el servicio: se agregó la clase HTMLReport y el case "html" en ReportFactory.create(). ReportService.generate() quedó idéntico.

6. Variante implementada: Static Factory

Variante — Cuándo usarla — Este TP

GoF clásico (jerarquía de creadores) — Cuando cada creador tiene lógica propia además de crear el producto — No aplica: solo varía el tipo de reporte

Static Factory — Cuando solo cambia el producto concreto, no el comportamiento del creador — Elegida

Elegí Static Factory porque la decisión es únicamente "qué subclase de Report instanciar". No hay comportamiento distinto en el creador que justifique subclases como PdfReportService, ExcelReportService, etc.

7. Factory Method vs Abstract Factory

Criterio — Factory Method (este caso) — Abstract Factory

Qué crea — Un producto con variantes (Report) — Familias de objetos relacionados

Ejemplo típico — Report en PDF/Excel/CSV/HTML — Botón + Input + Checkbox del mismo tema

Extensión — Nuevo formato → agregar en factory — Nueva familia → nueva fábrica abstracta

Complejidad — Baja, adecuada al problema — Innecesaria aquí

Conclusión: Abstract Factory resolvería un problema que no tenemos (familias coordinadas). Factory Method con Static Factory concentra la creación y deja al servicio limpio.

8. Justificación del patrón (máx. 10 líneas)

El acoplamiento estaba en la creación de objetos concretos dentro de ReportService. Cada formato nuevo contaminaba el servicio con if/elif de instanciación. Factory Method separa esa responsabilidad: el servicio solo opera sobre la interfaz Report y delega la decisión a ReportFactory. Esto permite agregar HTMLReport tocando únicamente la fábrica. Elegí Static Factory porque no necesitamos subclases de creador con comportamiento distinto.

9. Evidencia de ejecución

```
[PDF]    Reporte Mensual | {'producto': 'A', 'ventas': 120}, ... | Generado el 2026-08-30
[EXCEL]  Reporte Mensual | {'producto': 'A', 'ventas': 120}, ... | Generado el 2026-08-30
[CSV]    Reporte Mensual | {'producto': 'A', 'ventas': 120}, ... | Generado el 2026-08-30
[HTML]   Reporte Mensual | {'producto': 'A', 'ventas': 120}, ... | Generado el 2026-08-30
```

El formato HTML funciona sin modificar ReportService.generate().